

Universidade Tiradentes

Curso: Ciência da Computação

Integrantes Grupo 05:

Heberth Hugo De Andrade Santana

Gustavo Wagner Cruz Cunha

Lucas Silva

Relatório Técnico – Desenvolvimento do Compilador RoboLang

Aracaju-SE

Novembro de 2025

Integrantes Grupo 05:

Heberth Hugo De Andrade Santana

Gustavo Wagner Cruz Cunha

Lucas Silva

RoboLang

ATIVIDADE sobre o desenvolvimento do compilador da linguagem RoboLang, apresentada como requisito parcial de avaliação da disciplina **Compiladores**, ministrada pela Prof.^a **Layse Santos Souza**, no **2º semestre de 2025**.

Aracaju-SE

Novembro de 2025

Sumário

1. Introdução

2. Justificativa

3. Objetivos

3.1 Objetivo Geral

3.2 Objetivos Específicos

4. Metodologia

4.1 Planejamento e Organização do Projeto

4.2 Ferramentas e Tecnologias Utilizadas

4.3 Estruturação e Modularização do Código

4.4 Processo de Testes e Validação

4.5 Documentação e Apresentação Final

5. Resultados e Discussões

5.1 Tokens e Gramática

5.2 Análise Léxica

5.3 Análise Sintática

5.4 Análise Semântica

5.5 Geração de Código Intermediário (IR)

5.6 Execução e Simulação

5.7 Resultados de Testes

5.8 Discussão Geral

6. Considerações Finais

7. Referências

1. Introdução

O presente trabalho descreve o projeto de implementação de um compilador para a linguagem **RoboLang**, desenvolvido pelo Grupo 05 como parte das atividades da disciplina de **Compiladores**. RoboLang é uma linguagem de domínio específico projetada para descrever comandos de movimento e controle de um robô (por exemplo: mover, virar, velocidade, esperar, repetir). O objetivo do projeto é ter uma experiência prática e integrada das principais fases de construção de compiladores — desde a definição da gramática até a análise semântica, geração de uma representação intermediária (IR) e execução simulada dos comandos.

A motivação do projeto advém da necessidade de consolidar conceitos teóricos da disciplina (análise léxica e sintática, análise semântica, construção de árvores abstratas, geração de código intermediário e simulação) em um artefato funcional e modular. Para tanto, adotou-se uma abordagem orientada a objetos e ferramentas consolidadas de geração de analisadores: a gramática formal foi especificada em **RoboLang.g4** e o *tooling* utilizado para geração do lexer/parser foi o **ANTLR 4.13.1**, com backend em Python. A pilha de implementação inclui scripts e módulos em Python responsáveis pelo parse, pela construção da AST (Abstract Syntax Tree), pela análise semântica, pela conversão para IR (JSON/pseudo-código) e por um módulo de simulação/execução.

A execução prática do projeto foi organizada em entregas semanais. Na **Semana 01** implementou-se primeiramente o analisador semântico e a tabela de símbolos, com mecanismos de detecção e registro de erros (arquivo de log `erros_semanticos.log`). Em razão de exigência de qualidade e correção, a implementação inicial da semana 01 foi posteriormente refeita; as versões consolidadas das Semanas 01, 02 e 03 estão contempladas neste relatório. Na **Semana 02** foi desenvolvida a tradução das regras para uma representação intermediária (IR), com scripts como `convert_examples.py` que convertem exemplos `.robo` em estruturas JSON para facilitar inspeção e integração. Na **Semana 03** procedeu-se à integração dos módulos — parser, construtor de AST, analisador semântico e simulador — e à criação de testes (scripts de demonstração e testes automáticos) para validar programas válidos e detectar programas inválidos.

Do ponto de vista metodológico, o projeto privilegiou: (i) **separação de responsabilidades**, organizando cada etapa do compilador em classes e módulos independentes; (ii) **robustez na verificação semântica**, registrando erros semânticos sem interromper a análise do restante do programa; e (iii) **reprodutibilidade**, fornecendo exemplos (`exemple.robo`), scripts de demonstração (`demo_semantic.py`, `test_semantic.py`) e um `main.py` para demonstrar o parse e a árvore sintática. A documentação técnica e os artefatos de execução foram estruturados de modo a facilitar a avaliação, a replicação dos testes no ambiente de desenvolvimento (por exemplo, PyCharm e Vscod) e a inclusão de evidências (prints) das saídas obtidas.

Este relatório está organizado para apresentar, de forma sistemática, os objetivos do projeto, o desenvolvimento técnico por semanas (com descrição das classes e dos algoritmos empregados), os resultados dos testes e simulações, e as conclusões e recomendações para trabalhos futuros. Nos anexos constam instruções de execução, nomes dos arquivos e espaços reservados para capturas de tela das saídas que documentam a execução dos scripts principais.

2. Justificativa

O desenvolvimento do projeto **RoboLang** justifica-se pela necessidade de aplicar de forma prática e integrada os conceitos fundamentais de **construção de compiladores** estudados na disciplina, permitindo compreender de maneira concreta o ciclo completo de tradução de uma linguagem — desde a definição da gramática até a execução simulada de instruções. A criação de uma linguagem própria para controle de robôs proporciona um contexto didático e realista, no qual os estudantes podem visualizar de maneira tangível o resultado das etapas de análise léxica, sintática e semântica, bem como a geração e execução de código intermediário.

A escolha por uma linguagem de domínio específico voltada a comandos robóticos tem valor pedagógico por unir teoria e prática de forma lúdica e intuitiva: cada instrução representa uma ação concreta (mover, virar, esperar, repetir), o que facilita o entendimento dos mecanismos internos de um compilador e da relação entre código-fonte e execução. Além disso, esse domínio possibilita explorar diferentes níveis de abstração — desde o reconhecimento de tokens até a simulação do comportamento do robô —, demonstrando como um código simbólico se transforma em comportamento observável.

Outro fator que fundamenta a relevância deste projeto é a utilização do **ANTLR- 4.13.1**, uma ferramenta amplamente adotada em ambientes profissionais e acadêmicos para a geração de analisadores. Sua integração ao ambiente **Python 3** torna o processo mais acessível e flexível, incentivando boas práticas de engenharia de software, como modularização e separação de responsabilidades. Essa estrutura orientada a objetos foi essencial para garantir a manutenção e a evolução incremental do compilador, permitindo que cada fase (análise semântica, geração de IR, simulação) fosse desenvolvida, testada e refinada de forma independente e posteriormente integrada.

A execução semanal das tarefas proporcionou uma progressão gradual de complexidade, favorecendo o aprendizado contínuo e a consolidação dos conceitos estudados. A **Semana 01** introduziu o raciocínio sobre variáveis, tipos e escopos, além de reforçar a importância da tabela de símbolos e da detecção de erros semânticos.

Na **Semana 02**, a tradução das regras para uma **representação intermediária (IR)** consolidou a visão de compilação como transformação sucessiva de representações, facilitando a depuração e a análise estrutural do código. Já a **Semana 03** demonstrou a integração prática das fases, culminando na execução simulada dos comandos — etapa que evidenciou o papel do compilador como um sistema completo de interpretação e controle.

Portanto, o projeto RoboLang é justificado não apenas como uma atividade avaliativa, mas como uma **experiência formativa essencial**, que reforça os fundamentos teóricos da disciplina de Compiladores, estimula a autonomia na resolução de problemas técnicos e incentiva o desenvolvimento de competências práticas em programação estruturada, modelagem de linguagens e simulação computacional.

2. Objetivos

O projeto **RoboLang** tem como propósito principal desenvolver um compilador funcional e modular, capaz de interpretar e simular comandos de movimentação de um robô a partir de uma linguagem própria. A atividade foi planejada para promover o entendimento prático das fases que compõem o processo de compilação — análise semântica, geração de representação intermediária e simulação —, integrando conceitos teóricos da disciplina de Compiladores com a aplicação direta em um sistema orientado a objetos.

O trabalho foi estruturado em etapas semanais, permitindo a construção progressiva de cada componente do compilador. A execução dessas fases teve como meta consolidar o aprendizado sobre análise semântica e tratamento de erros, geração de código intermediário (IR) e integração final entre os módulos, culminando em uma simulação executável dos comandos da linguagem RoboLang.

3.1 Objetivos Geral

Desenvolver um **compilador completo para a linguagem RoboLang**, capaz de realizar análise semântica, gerar uma representação intermediária (IR) e executar de forma simulada os comandos definidos na linguagem. O projeto visa demonstrar a aplicação prática dos princípios de compiladores, utilizando o **ANTLR 4.13.1** como ferramenta de base para geração automática dos analisadores léxico e sintático e o **Python 3** como linguagem de implementação e integração dos módulos.

3.2 Objetivos Específicos

1. **Implementar a análise semântica** da linguagem RoboLang, identificando variáveis, tipos e escopos, além de registrar erros semânticos e documentar as soluções adotadas.
2. **Construir uma tabela de símbolos** para armazenar e gerenciar as variáveis e estruturas da linguagem, assegurando consistência entre declarações e usos.
3. **Gerar uma representação intermediária (IR)** em formato **JSON ou pseudo-código**, demonstrando a conversão das instruções RoboLang em estruturas manipuláveis e legíveis por máquina.
4. **Validar a tradução e a IR gerada** por meio de exemplos práticos (mínimo de três casos), comprovando a coerência entre as regras da gramática e a estrutura intermediária resultante.
5. **Implementar um simulador de execução**, capaz de interpretar a sequência das instruções RoboLang a partir da IR, reproduzindo o comportamento dos comandos de movimento e controle.
6. **Integrar todas as fases do compilador** (léxica, sintática, semântica e de simulação), garantindo o fluxo contínuo entre leitura do código-fonte, análise e execução.
7. **Organizar o código em classes e módulos independentes**, aplicando o paradigma da programação orientada a objetos para facilitar manutenção, depuração e reuso em futuras versões.
8. **Documentar tecnicamente o desenvolvimento do compilador**, detalhando as etapas realizadas, os resultados obtidos e as ferramentas utilizadas, conforme as normas da ABNT.

4. Metodologia

O desenvolvimento do projeto foi conduzido segundo uma metodologia incremental, prática e orientada a objetos, baseada nas etapas fundamentais da construção de um compilador. Cada fase foi planejada, implementada e testada de forma independente, assegurando a integração progressiva entre os módulos e a consistência do sistema.

O projeto foi desenvolvido na linguagem Python 3, utilizando o ANTLR 4.13.1 para a definição da gramática e geração dos analisadores léxico e sintático. A abordagem adotada privilegiou a modularização do código, a clareza estrutural e o registro contínuo dos resultados, promovendo uma evolução iterativa e controlada do compilador.

4.1 Planejamento e Organização do Projeto

O projeto foi estruturado em quatro semanas de desenvolvimento, cada uma com objetivos específicos que correspondem a etapas fundamentais do processo de compilação:

- Semana 01 (Revisada) – Implementação do analisador semântico e da tabela de símbolos, responsáveis por identificar variáveis, tipos e escopos, além de registrar erros semânticos em log.
- Semana 02 – Desenvolvimento da representação intermediária (IR), convertendo as regras da linguagem RoboLang em estruturas JSON ou pseudo-código e validando os resultados por meio de exemplos práticos.
- Semana 03 – Criação do simulador de execução, responsável por interpretar as instruções RoboLang a partir da IR e executar as tarefas na ordem correta, integrando todas as fases anteriores do compilador.
- Semana 04 (13/11) – Documentação e Apresentação Final, com a elaboração da documentação técnica em conformidade com as normas da ABNT, reunindo todas as etapas do projeto (tokens, gramática, análise léxica, sintática e semântica, geração de código, execução, simulação e resultados de testes). Nesta fase, também foi preparada a apresentação final em vídeo, demonstrando o funcionamento completo do compilador RoboLang.

4.2 Ferramentas e Tecnologias Utilizadas

ANTLR 4.13.1 – Geração automática dos analisadores léxico e sintático com base na gramática RoboLang.g4;

Python 3 – Linguagem principal para a implementação dos módulos do compilador (análise, geração de IR e simulação);

PyCharm – Ambiente de desenvolvimento utilizado para programação, testes e depuração;

JSON – Formato adotado para representar a estrutura intermediária dos programas RoboLang;

Scripts de validação – `test_semantic.py`, `convert_exemples.py`, `demo_semantic.py` e `main.py`, utilizados para execução e testes práticos.

4.3 Estruturação e Modularização do Código

O código do compilador foi desenvolvido com base no paradigma de programação orientada a objetos, permitindo a separação clara das fases de compilação e facilitando manutenção e expansão futura. As principais classes e módulos foram:

Lexer e Parser (ANTLR): responsáveis pela análise léxica e sintática da linguagem;

AST Builder: construtor da árvore sintática abstrata (AST) a partir da árvore de análise gerada pelo parser;

Analizador Semântico: verifica escopos, tipos e erros de coerência nas instruções;

Gerador de IR: converte a AST para uma representação intermediária (JSON/pseudo-código);

Simulador: interpreta e executa os comandos do robô, demonstrando o comportamento final do programa.

4.4 Processo de Testes e Validação

Durante o desenvolvimento, foram realizados testes incrementais para garantir a consistência e a robustez de cada fase do compilador. Os scripts de teste foram executados no PyCharm, com registro das saídas de console para documentação posterior.

As validações contemplaram:

- Testes de detecção de erros semânticos e verificação de escopo;
- Testes de conversão de código-fonte para representação intermediária (IR);
- Testes de integração entre análise, geração e simulação;
- Execuções completas de programas RoboLang, com análise das saídas esperadas;
- Registro das evidências e prints das execuções para inclusão no relatório.

4.5 Documentação e Apresentação Final

Na Semana 04 (13/11), foram realizadas as atividades de documentação e apresentação final do compilador RoboLang. Nesta fase, o grupo reuniu todos os artefatos técnicos produzidos — tokens, gramática, arquivos gerados pelo ANTLR, scripts de análise, IRs resultantes, simulações e testes —, estruturando-os de acordo com as normas da ABNT.

Também foi elaborada uma apresentação final em formato de vídeo curto, demonstrando a execução prática do compilador e o funcionamento integrado de suas fases. O vídeo teve como objetivo apresentar de forma clara e objetiva os resultados obtidos, os desafios enfrentados e a importância da

aplicação dos conceitos teóricos da disciplina de Compiladores no desenvolvimento do sistema RoboLang.

5. RESULTADOS E DISCUSSÕES

O desenvolvimento do compilador RoboLang resultou na construção de um sistema completo e funcional, capaz de processar e interpretar uma linguagem específica voltada para controle robótico. Ao longo das quatro semanas de trabalho, foram implementadas todas as fases fundamentais de um compilador — desde a definição dos tokens e da gramática, até a simulação final da execução dos programas.

A seguir, são apresentados os principais resultados obtidos, acompanhados de discussões técnicas sobre cada módulo e com espaços reservados para anexação de prints, códigos e diagramas que documentam o funcionamento real do compilador.

5.1 Tokens e Gramática

A base da linguagem RoboLang foi especificada no arquivo RoboLang.g4, utilizando a ferramenta ANTLR 4.13.1 para a definição dos tokens (símbolos léxicos) e das regras gramaticais que determinam a estrutura sintática dos programas.

Entre os tokens definidos, destacam-se as palavras-chave principais da linguagem:

- ROBO, MOVER, VIRAR, VELOCIDADE, ESPERAR, REPETIR, além dos identificadores (ID), números (NUMBER), eixos (EIXO), e símbolos de controle (LBACE, RBACE, etc.).
- O conjunto de regras da gramática foi desenvolvido para garantir a legibilidade e a coerência estrutural da linguagem, permitindo a criação de blocos de comandos dentro de um robô definido.

```

1  grammar RoboLang;
2  programa      : ROBO ID LBRACE listaComandos RBRACE EOF;
3  listaComandos : comando* ;
4  comando       : mover
5                  | virar
6                  | velocidade
7                  | esperar
8                  | repetir
9                  ;
10
11 mover          : MOVER EIXO distancia ;
12 virar          : VIRAR angulo ;
13 velocidade     : VELOCIDADE NUMBER ;
14 esperar        : ESPERAR NUMBER MS? ;
15 repetir        : REPETIR NUMBER LBRACE listaComandos RBRACE ;
16 distancia      : NUMBER unidade ;
17 angulo         : NUMBER GRAUS ;
18 unidade        : CM | M ;
19
20 ROBO           : 'robo';
21 MOVER          : 'mover';
22 VIRAR          : 'virar';
23 VELOCIDADE     : 'velocidade';
24 ESPERAR        : 'esperar';
25 REPETIR        : 'repetir';
26 GRAUS         : 'graus';
27 CM             : 'cm';
28 M              : 'm';
29 MS             : 'ms';
30 EIXO           : 'x' | 'y' | 'z';
31 LBRACE         : '{';
32 RBRACE         : '}';
33 ID             : [a-zA-Z][a-zA-Z0-9]*;
34 NUMBER         : [0-9]+ ('.' [0-9]+)?;
35 WS             : [ \t\r\n]+ -> skip;
36

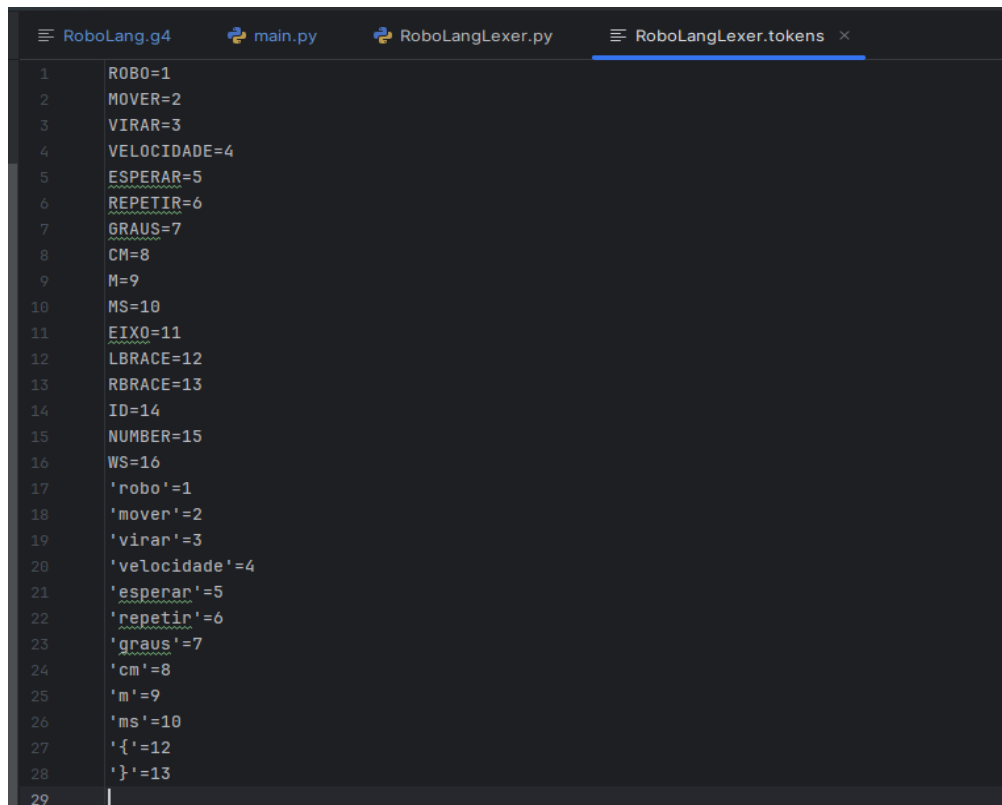
```

[Figura 1 – Estrutura geral da gramática RoboLang e tokens definidos]

5.2 Análise Léxica

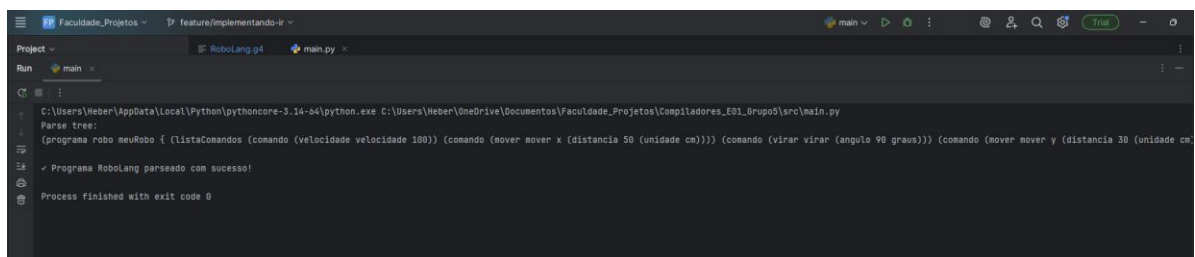
A análise léxica é responsável por converter o código-fonte em uma sequência de tokens reconhecidos pelo compilador. Essa etapa é executada automaticamente pelos arquivos RoboLangLexer.py e RoboLangParser.py, gerados pelo ANTLR a partir da gramática.

Durante os testes, o lexer identificou corretamente todos os tokens da linguagem, rejeitando entradas inválidas e reportando erros léxicos quando encontrados. O uso do ANTLR permitiu abstrair as complexidades do processo, garantindo um reconhecimento eficiente e preciso dos símbolos.



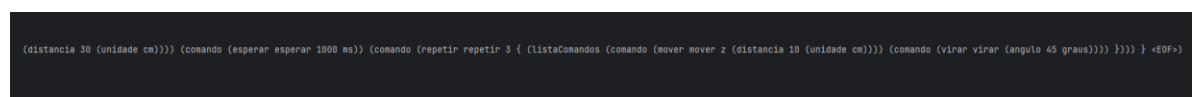
```
1 ROBO=1
2 MOVER=2
3 VIRAR=3
4 VELOCIDADE=4
5 ESPERAR=5
6 REPETIR=6
7 GRAUS=7
8 CM=8
9 M=9
10 MS=10
11 EIXO=11
12 LBRACE=12
13 RBRACE=13
14 ID=14
15 NUMBER=15
16 WS=16
17 'robo'=1
18 'mover'=2
19 'virar'=3
20 'velocidade'=4
21 'esperar'=5
22 'repetir'=6
23 'graus'=7
24 'cm'=8
25 'm'=9
26 'ms'=10
27 '{'=12
28 '}'=13
29
```

[Figura 2 –Print código de teste léxico]



```
Project: Faculdade_Projetos
Run: main
C:\Users\Heber\AppData\Local\Python\pythoncore-3.14-64\python.exe C:\Users\Heber\OneDrive\Documents\Faculdade_Projetos\compiladores_E01_grupo5\src\main.py
Parse tree:
(programa robo meuRobo { (listaComandos (comando (velocidade velocidade 100)) (comando (mover mover x (distancia 50 (unidade cm)))) (comando (virar virar (angulo 90 graus))) (comando (mover mover y (distancia 30 (unidade cm)))) (comando (esperar esperar 1000 ms)) (comando (repetir repetir 3 { (listaComandos (comando (mover mover z (distancia 10 (unidade cm)))) (comando (virar virar (angulo 45 graus)))) } } } <EOF>)
✓ Programa RoboLang parseado com sucesso!
Process finished with exit code 0
```

[Figura 3 – Saída 1º parte de execução do lexer demonstrando a tokenização no console]



```
(distancia 30 (unidade cm)))) (comando (esperar esperar 1000 ms)) (comando (repetir repetir 3 { (listaComandos (comando (mover mover z (distancia 10 (unidade cm)))) (comando (virar virar (angulo 45 graus)))) } } } <EOF>)
```

[Figura 3.1 – Saída 2º parte de execução do lexer demonstrando a tokenização no console]

5.3 Análise Sintática

A análise sintática verificou a organização dos tokens segundo as regras da gramática. O arquivo RoboLangParser.py gerou a árvore de análise (Parse Tree), a qual representa a estrutura hierárquica do programa RoboLang.

Essa etapa foi validada através do script main.py, que realiza o parse de um programa exemplo e exibe sua árvore sintática. A correta geração da árvore confirmou que a gramática implementada cobre todos os construtos esperados, como blocos de comandos e estruturas de repetição.

```
19 def main(): 1 usage  Gustavo
20     # Criar o input stream
21     input_stream = InputStream(input_text)
22
23     # Criar o lexer
24     lexer = RoboLangLexer(input_stream)
25
26     # Criar o token stream
27     token_stream = CommonTokenStream(lexer)
28
29     # Criar o parser
30     parser = RoboLangParser(token_stream)
31
32     # Parsear o programa
33     tree = parser.programa()
34
35     # Imprimir a árvore
36     print("Parse tree:")
37     print(tree.toStringTree(recog=parser))
38     print("\n✓ Programa RoboLang parseado com sucesso!")
```

[Figura 3 – Árvore sintática gerada para o programa de exemplo em RoboLang]

5.4 Análise Semântica

Na análise semântica, foram introduzidos mecanismos de verificação de escopo, tipo e coerência entre as instruções, implementados no módulo analisador_semantico.py. Essa etapa foi essencial para identificar erros como variáveis não declaradas, comandos fora de contexto e inconsistências de tipo.

O sistema também registrou os erros semânticos detectados em um arquivo de log para consulta posterior. Durante os testes, observou-se que o analisador semântico conseguia não apenas detectar os erros, mas também continuar a execução e listar todos os problemas encontrados, melhorando a confiabilidade da depuração.

```

7 class SemanticError(Exception): 7 usages Gustavo
8     """Base class for semantic errors."""
9     def __init__(self, message: str, node=None): Gustavo
10         self.message = message
11         self.node = node
12         super().__init__(message)
13
14 class InvalidValueError(SemanticError): 6 usages Gustavo
15     """Raised when a numeric value is invalid (e.g., negative when positive required)."""
16     pass
17
18 class InvalidUnitError(SemanticError): 2 usages Gustavo
19     """Raised when an invalid unit is used."""
20     pass
21
22 class InvalidAngleError(SemanticError): 2 usages Gustavo
23     """Raised when an angle is outside valid range."""
24     pass
25
26 class InvalidRepeatCountError(SemanticError): 2 usages Gustavo
27     """Raised when repeat count is invalid."""
28     pass
29
30 class SemanticErrorCollector: 3 usages Gustavo
31     """Collects multiple semantic errors during analysis."""
32     def __init__(self): Gustavo
33         self.errors: List[SemanticError] = []
34
35     def add_error(self, error: SemanticError): 1 usage Gustavo
36         self.errors.append(error)
37
38     def has_errors(self) -> bool: 2 usages Gustavo
39         return len(self.errors) > 0
40
41     def get_error_messages(self) -> List[str]: 2 usages Gustavo
42         return [e.message for e in self.errors]
43
44     def raise_if_errors(self): 1 usage Gustavo
45         if self.has_errors():
46             msg = "Semantic errors found:\n" + "\n".join(f" - {e}" for e in self.get_error_messages())
47             raise SemanticError(msg)

```

[Figura 4 –analizador semântico mensagens de erro no console]

```

PROBLEMAS 8 SAÍDA CONSOLE DE DEPURACÃO TERMINAL PORTAS GITLENS DEVOB
PS C:\Users\gwgus\Documents\Compiladores_E01_Grupo5> python -m src.test_semantic
Program: test
Commands: 1

❌ INVÁLIDO - Erros semânticos encontrados:
  • Angulo deve estar entre 0 e 360 graus, obteve 400.0

=====
Analizando: velocidade_invalida. robo
=====
line 2:13 token recognition error at: '-'

```

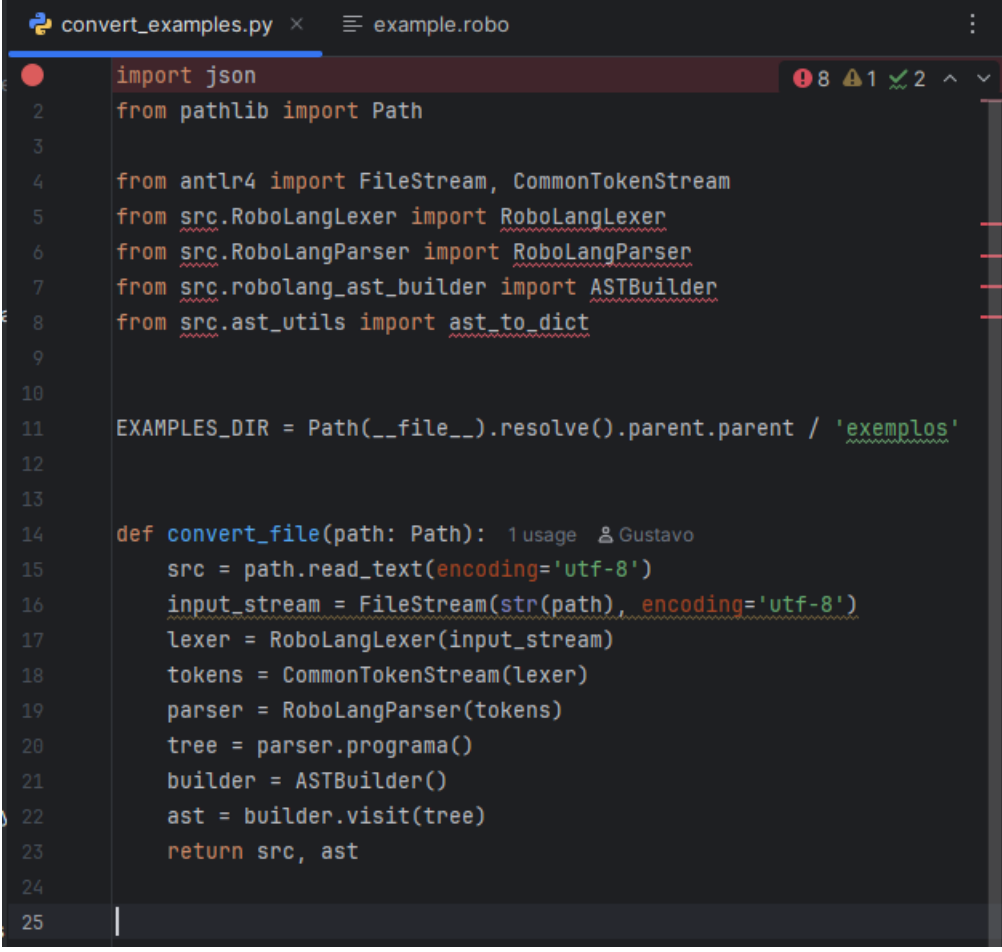
[Figura 4.1 – Saída do analisador semântico mostrando mensagens de erro no console]

5.5 Geração de Código Intermediário (IR)

A geração de código intermediário (IR) foi implementada na segunda fase do projeto, utilizando scripts como `convert_exemples.py` e `demo_semantic.py`. O objetivo dessa etapa foi traduzir

a árvore sintática abstrata (AST) para uma estrutura JSON ou pseudo-código, representando as instruções RoboLang em um formato genérico e interpretável.

Essa representação intermediária permitiu validar a coerência das instruções e facilitou a posterior execução no simulador. Durante os testes, foram produzidos múltiplos exemplos (mínimo de três) de programas RoboLang convertidos com sucesso para IR.



```
convert_examples.py x example.roboto
import json
from pathlib import Path

from antlr4 import FileStream, CommonTokenStream
from src.RoboLangLexer import RoboLangLexer
from src.RoboLangParser import RoboLangParser
from src.robolang_ast_builder import ASTBuilder
from src.ast_utils import ast_to_dict

EXAMPLES_DIR = Path(__file__).resolve().parent.parent / 'exemplos'

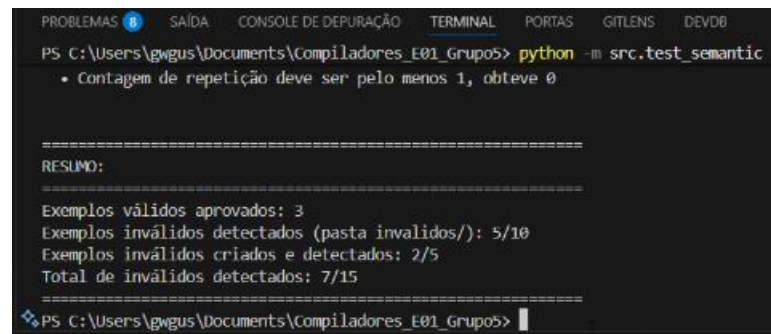
def convert_file(path: Path): 1 usage Gustavo
    src = path.read_text(encoding='utf-8')
    input_stream = FileStream(str(path), encoding='utf-8')
    lexer = RoboLangLexer(input_stream)
    tokens = CommonTokenStream(lexer)
    parser = RoboLangParser(tokens)
    tree = parser.programa()
    builder = ASTBuilder()
    ast = builder.visit(tree)
    return src, ast
```

[Figura 5 – Exemplo de saída JSON da representação intermediária (IR) de um programa RoboLang]

5.6 Execução e Simulação

Na terceira etapa, implementou-se o simulador de execução, integrado às fases anteriores, responsável por interpretar a representação intermediária e executar os comandos em sequência. O simulador foi projetado para simular o comportamento de um robô, processando ações como movimentação, rotação, espera e repetição.

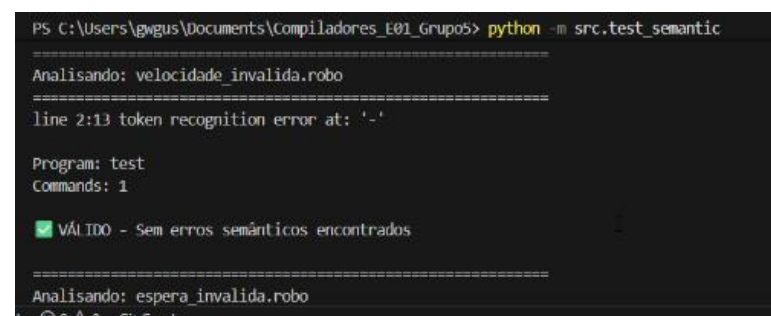
O módulo de simulação foi validado por meio do script `test_semantic.py`, que testou programas válidos e inválidos, assegurando a execução correta e a detecção de erros. Essa fase também serviu como integração final de todos os componentes, confirmando o funcionamento completo do compilador.



```
PROBLEMAS 6 SAÍDA CONSOLE DE DEPURAÇÃO TERMINAL PORTAS GITLENS DEVDB
PS C:\Users\gwigus\Documents\Compiladores_E01_Grupo5> python -m src.test_semantic
• Contagem de repetição deve ser pelo menos 1, obteve 0

=====
RESUMO:
=====
Exemplos válidos aprovados: 3
Exemplos inválidos detectados (pasta invalidos/): 5/10
Exemplos inválidos criados e detectados: 2/5
Total de inválidos detectados: 7/15
=====
PS C:\Users\gwigus\Documents\Compiladores_E01_Grupo5>
```

[Figura 6.0 – Resultado da simulação executando comandos RoboLang válidos e inválidos]



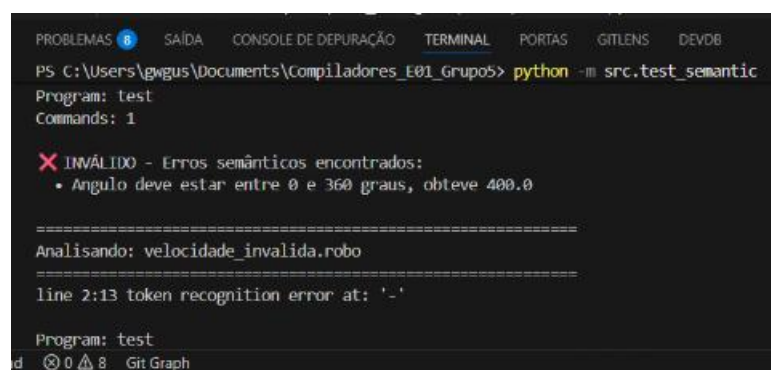
```
PS C:\Users\gwigus\Documents\Compiladores_E01_Grupo5> python -m src.test_semantic
=====
Analisando: velocidade_invalida.rob
=====
line 2:13 token recognition error at: '-'

Program: test
Commands: 1

✅ VÁLIDO - Sem erros semânticos encontrados

=====
Analisando: espera_invalida.rob
=====
```

[Figura 6.1 – Resultado da simulação executando comandos RoboLang válidos e inválidos]



```
PROBLEMAS 6 SAÍDA CONSOLE DE DEPURAÇÃO TERMINAL PORTAS GITLENS DEVDB
PS C:\Users\gwigus\Documents\Compiladores_E01_Grupo5> python -m src.test_semantic
Program: test
Commands: 1

❌ INVÁLIDO - Erros semânticos encontrados:
• Angulo deve estar entre 0 e 360 graus, obteve 400.0

=====
Analisando: velocidade_invalida.rob
=====
line 2:13 token recognition error at: '-'

Program: test
```

[Figura 6.2 – Resultado da simulação executando comandos RoboLang válidos e inválidos]

5.7 Resultados de Testes

Os testes foram realizados de forma incremental, validando individualmente as fases e, posteriormente, a integração total.

- Entre os resultados mais relevantes estão:
- Reconhecimento preciso dos tokens e regras gramaticais;
- Criação correta da árvore sintática para todos os exemplos testados;
- Identificação e registro confiável de erros semânticos;
- Geração consistente de código intermediário (IR) no formato JSON;
- Execução e simulação bem-sucedidas de programas válidos;

Relatórios de saída detalhados e reproduzíveis no ambiente PyCharm.

5.8 Discussão Geral

Os resultados obtidos confirmam a viabilidade técnica e didática da linguagem RoboLang como ferramenta educacional para o ensino de compiladores. O uso do ANTLR 4.13.1 mostrou-se eficiente na geração dos analisadores, reduzindo o tempo de desenvolvimento e permitindo foco nas fases semântica e de execução. A abordagem incremental possibilitou uma integração coerente entre os módulos, além de fornecer uma visão clara do fluxo interno de um compilador moderno.

A implementação da simulação final evidenciou o funcionamento prático das instruções RoboLang, tornando o aprendizado mais concreto e demonstrando, de forma visual e funcional, a aplicação dos conceitos teóricos de análise e tradução de linguagens formais.

6. Considerações Finais

O desenvolvimento do compilador RoboLang representou um processo de aprendizagem intenso e progressivo, marcado por desafios significativos em cada uma das etapas. A retrospectiva do projeto demonstra que as maiores dificuldades encontradas pelo grupo surgiram ainda na **Semana 01**, quando foi implementado um analisador semântico totalmente manual, sem o uso de ferramentas formais como ANTLR.

Essa primeira abordagem, baseada em código próprio para tabela de símbolos, escopos, verificação de tipos e registro de erros em log — conforme o arquivo original da Semana 01 — funcionava apenas para testes isolados e não seguia o fluxo completo de um compilador real. A ausência de análise léxica e sintática formal tornou impossível expandir essa base para as fases seguintes. Por esse motivo, essa versão precisou ser **abandonada**, exigindo que todo o trabalho fosse reiniciado com ANTLR na Semana 02.

Essa mudança de abordagem gerou um novo conjunto de dificuldades. A necessidade de compreender a estrutura gerada pelo ANTLR — tokens, árvore sintática, regras da gramática e visitantes — impactou diretamente todas as semanas subsequentes. A integração entre o novo analisador sintático, o ASTBuilder, a tabela de símbolos, a geração de IR e o simulador exigiu uma reestruturação completa do projeto.

A migração do código manual da Semana 01 para uma arquitetura baseada em ANTLR tornou-se, de fato, **a parte mais difícil de todo o projeto**, pois exigiu:

- reescrever a lógica de escopos;
- adaptar a verificação de tipos para trabalhar com a AST real;
- redefinir a forma como os comandos RoboLang eram interpretados;
- garantir que a nova estrutura fosse compatível com a geração de JSON na Semana 02;
- ajustar o simulador da Semana 03 para consumir a IR corretamente.

Além disso, houve dificuldades na definição e interpretação da gramática, especialmente na tradução das regras para a AST, o que resultou em erros constantes até que os métodos do visitante fossem devidamente ajustados. A cada pequena mudança na gramática, era necessário revisar o ASTBuilder, o gerador de IR e o simulador, tornando o processo altamente interdependente e sensível a falhas.

A integração final — exigida na Semana 03 — mostrou-se particularmente trabalhosa porque os módulos funcionavam bem isoladamente, mas apresentavam inconsistências quando operavam no fluxo completo. A simulação também revelou erros semânticos e casos especiais que precisaram ser tratados manualmente, como comandos inválidos (“girarr”) ou eixos inexistentes (“w”).

Mesmo com todas essas dificuldades, o grupo conseguiu concluir um compilador funcional, capaz de:

- interpretar arquivos .robo;
- analisar tokens e regras sintáticas via ANTLR;
- construir uma AST estruturada;
- gerar uma representação intermediária (IR) em JSON;
- e executar uma simulação coerente com comandos válidos e inválidos.

O projeto, embora desafiador desde a primeira versão até a implementação final, proporcionou um aprendizado profundo sobre os conceitos de compiladores e demonstrou na prática a complexidade envolvida na construção de uma linguagem formal, especialmente quando se enfrenta a necessidade de refatorar e reconstruir módulos inteiros ao longo do processo.

7. Referência

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. **Compiladores: Princípios, Técnicas e Ferramentas**. 2. ed. São Paulo: Pearson, 2008.

HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. **Introdução à Teoria de Autômatos, Linguagens e Computação**. 3. ed. São Paulo: Pearson, 2011.

ANTLR Tool. **ANTLR 4 Documentation – Version 4.13.1**. Disponível em: <https://www.antlr.org>. Acesso em: 10 nov. 2025.

PARR, Terence. **The Definitive ANTLR 4 Reference**. 2. ed. Dallas: Pragmatic Bookshelf, 2013.

ECMA International. **The JSON Data Interchange Format**. ECMA-404, 2nd Edition, 2017. Disponível em: <https://www.ecma-international.org/publications-and-standards/standards/ecma-404/>. Acesso em: 10 nov. 2025.

PYTHON Software Foundation. **Python Language Reference**. Disponível em: <https://docs.python.org/3/>. Acesso em: 10 nov. 2025.

AHO, Alfred V.; ULLMAN, Jeffrey D. **Principles of Compiler Design**. Reading: Addison-Wesley, 1977.

SIPER, Michael. **Introduction to the Theory of Computation**. 3. ed. Boston: Cengage Learning, 2012.

ISO/IEC. **ISO/IEC 14977:1996 – Extended Backus–Naur Form (EBNF) Specification**. International Organization for Standardization, 1996.

W3C. **W3C JSON-LD 1.1 Specification**. Disponível em: <https://www.w3.org/TR/json-ld11>. Acesso em: 10 nov. 2025.